42 Warsaw Campus Dashboard

API Research and Data Strategy

Endpoints, refresh strategy, data handling and service reliability

Project Documentation

Prepared by ***eeravci & mafzal***

1. Solution Overview

The 42 Warsaw Campus Dashboard shows live and historical campus data. It is displayed on a screen in the Social Space and can also be opened as a web page. It helps students and staff quickly see what is happening on campus.

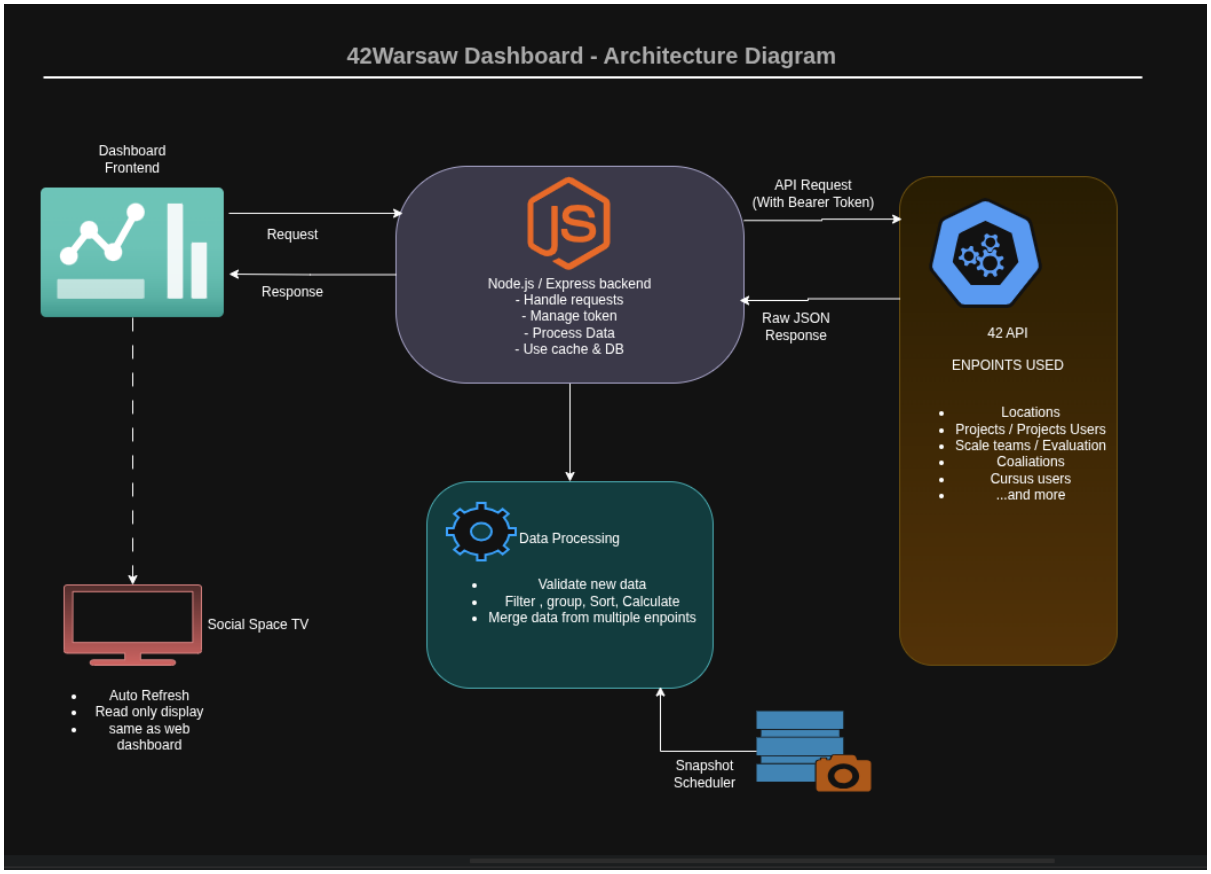
The dashboard gets information from the official 42 API and changes the raw data into simple visuals, such as:

- Current online students
- Cluster occupancy
- Most-used computers
- Weekly student activity
- Recently passed projects
- Top evaluators
- Most evaluated students
- Coalition standings
- Black Hole warnings
- Returning students
- Night Owl and Early Bird rankings

So, here's how it works: our frontend doesn't talk to the 42 API directly. Instead, it sends requests to our own backend, which then takes care of communicating with the 42 API.

This means the frontend only ever interacts with our backend, and gets data that's already been processed and is ready to use.

Architecture



Design principle: The backend manages authentication, pagination, filtering, caching and analytics. The frontend only receives processed, dashboard-ready JSON.

2. Authentication

The 42 API uses OAuth 2.0 for authentication. Because the dashboard doesn't represent a specific logged-in user, it uses the client-credentials grant to authenticate with the backend. This means the dashboard acts on its own, without being tied to a particular user's account, and it uses its own credentials to access the API.

Token Endpoint

```
POST https://api.intra.42.fr/oauth/token
```

Request Body

```
grant_type=client_credentials

client_id=CLIENT_ID

client_secret=CLIENT_SECRET
```

When you use the 42 API, the backend generates a special token called an access token. This token is sent along with every request you make to the API.

```
Authorization:  Bearer ACCESS_TOKEN
```

Authentication Rules

- CLIENT_ID and CLIENT_SECRET must be stored in environment variables.
- The client secret must never be sent to the frontend.
- The server should keep using the token until it's almost expired.
- When the token expires, the backend should request a new one automatically.
- A failed API request caused by an expired token should be retried only once after token renewal.

Example Environment Variables

```
CLIENT_ID=

CLIENT_SECRET=

 DEFAULT_CAMPUS_ID=

DEFAULT_CURSUS_ID=21
```

3. API Endpoints

This section documents every 42 API endpoint the dashboard uses. Each entry lists the HTTP method, the path, what the endpoint returns, and how the dashboard uses it. A summary table is provided first, followed by full detail for every endpoint, grouped by topic.

3.0 Endpoint Summary

Endpoint	Method	Data returned	Dashboard use	Refresh
/v2/project_sessions/:id	GET	One project session	Normalizing versions	On demand
/v2/scale_teams	GET	Evaluation records	Evaluator / evaluation stats	10-30 min
/v2/users/:user_id/scale_teams/as_corrector	GET	Evaluations given	Top evaluators	On demand
/v2/users/:user_id/scale_teams/as_corrected	GET	Evaluations received	Most evaluated students	On demand
/v2/projects/:project_id/scale_teams	GET	Evaluations for a project	Project evaluation patterns	On demand
/v2/scale_teams/:id	GET	One evaluation record	Debugging	On demand
/v2/coalitions	GET	Coalition definitions	Names, logos, standings	Hourly
/v2/blocs/:bloc_id/coalitions	GET	Coalitions in one bloc	Warsaw coalition set	Hourly
/v2/users/:user_id/coalitions	GET	A user's coalition	Coalition badge on profile	Daily
/v2/coalitions_users	GET	User-coalition relationships	Membership, contribution	Hourly / daily
/v2/coalitions/:coalition_id/coalitions_users	GET	Members of one coalition	Coalition member lists	On demand
/v2/users/:user_id/coalitions_users	GET	One user's coalition record	Contribution verification	On demand

Endpoint	Method	Data returned	Dashboard use	Refresh
/v2/project_sessions/:id	GET	One project session	Normalizing versions	On demand
/v2/scale_teams	GET	Evaluation records	Evaluator / evaluation stats	10-30 min
/v2/users/:user_id/scale_teams/as_corrector	GET	Evaluations given	Top evaluators	On demand
/v2/users/:user_id/scale_teams/as_corrected	GET	Evaluations received	Most evaluated students	On demand
/v2/projects/:project_id/scale_teams	GET	Evaluations for a project	Project evaluation patterns	On demand
/v2/scale_teams/:id	GET	One evaluation record	Debugging	On demand
/v2/coalitions	GET	Coalition definitions	Names, logos, standings	Hourly
/v2/blocs/:bloc_id/coalitions	GET	Coalitions in one bloc	Warsaw coalition set	Hourly
/v2/users/:user_id/coalitions	GET	A user's coalition	Coalition badge on profile	Daily
/v2/coalitions_users	GET	User-coalition relationships	Membership, contribution	Hourly / daily
/v2/coalitions/:coalition_id/coalitions_users	GET	Members of one coalition	Coalition member lists	On demand
/v2/users/:user_id/coalitions_users	GET	One user's coalition record	Contribution verification	On demand

3.1 Campuses

List campuses

GET /v2/campus

Returns the list of 42 campuses.

Use it for:

- Find the Warsaw campus ID.
- Avoid hardcoding the campus ID without verification.
- Confirm campus name and metadata.

Suggested refresh: Daily or weekly

Get one campus

```
GET /v2/campus/:i
```

Returns information about one campus.

Use it for:

- Confirm that the selected ID represents Warsaw.
- Retrieve campus-specific information.

Suggested refresh: Daily

Campus statistics

```
GET /v2/campus/:campus_id/stats
```

This endpoint may offer statistics for the entire campus, which are supported by the API. It's essential to only use this endpoint for fields that have been verified and are returned. We should not assume that it contains every single metric that the dashboard needs. Instead, we need to make sure that the fields we use are available and have been confirmed to work correctly.

Suggested refresh: Hourly

3.2 Campus Students

```
GET /v2/campus/:campus_id/users
```

Returns users connected to the selected campus.

Use it for:

- Warsaw student population.
- User login.
- User ID.
- Profile image.
- Student status.
- Staff or alumni filtering.
- Joining student data with projects, locations and cursus information.

Suggested refresh: Every 6-24 hours

Pagination parameters:

page[size] page[number]

Basic user information does not need minute-by-minute updates.

3.3 Users

One user

GET /v2/users/:id

Returns detailed information about a specific user.

Use it for:

- Profile image.
- Cursus memberships.
- Current level.
- Campus information.
- Black Hole date where provided through the cursus record.
- Additional user details needed by profile cards.

Suggested refresh: On demand or cached for several hours

Authenticated user

GET /v2/me

This function gives you the user who has been authenticated, which is useful when you're dealing with OAuth flows that need the user's permission. However, if you're working on a backend that only uses the client-credentials grant, you might not find this endpoint very useful. Also, keep in mind that you shouldn't rely on this for analytics that cover the entire campus.

3.4 Cursus Users

All cursus-user records

GET /v2/cursus_users

This function gives you information about a user's membership in a cursus. It can show you things like the user's ID, their cursus, what level they're at, their grade, when they started, when they finished, and if they got stuck in a "Black Hole" - that's when someone stops making progress. You get all this data when it's available.

Use it for: Student progress.

- Level rankings.
- Active cursus membership.
- Black Hole monitoring.

Suggested refresh: Every 6-12 hours

One user's cursus records

GET /v2/users/:user_id/cursus_users

Returns all cursus records for one user.

Use it for:

- Find the correct 42cursus record.
- Retrieve the student's level.
- Retrieve blackholed_at when available.
- Avoid using the wrong cursus.

Important: Never assume that cursus_users[0] is the main cursus. Match the record using the configured cursus ID.

Users in one cursus

```
GET /v2/cursus/:cursus_id/cursus_users
```

Returns users belonging to a specific cursus.

Use it for:

- Retrieving the main 42 curriculum population.
- Filtering out unrelated cursus records.
- Progress and Black Hole analytics.

For the main curriculum, use the configured cursus ID, which is usually set by default.

```
21
```

This value must not be hardcoded in multiple files. It should be stored in configuration.

One cursus-user relationship

```
GET /v2/cursus_users/:id
```

Returns one specific cursus-user relationship. Used when detailed information about one cursus record is required.

Note:

```
cursus_user.id is not the same as user.id
```

Cursus-user graph data

GET /v2/cursus_users/graph/on/:field/by/:interval

Returns grouped graph data when the selected field and interval are supported. This endpoint should only be used after testing which fields and intervals it accepts. The dashboard must not depend entirely on this endpoint, because custom historical analytics may still require local snapshots.

3.5 Locations and Cluster Activity

Campus locations

GET /v2/campus/:campus_id/locations

This feature gives you the details of all the location sessions for a specific campus. Some key things you might want to know about these sessions include who the user is, the name of the host or workstation, when the session started and ended, and which campus it's for.

Use it for:

- Students currently online.
- Live occupied seats.
- Session duration.
- Weekly campus-time rankings.
- Cluster occupancy.
- Seat popularity.
- Returning students.
- Night Owls.
- Early Birds.

Suggested refresh: Every 1-5 minutes for live occupancy

All result pages must be fetched, not only the first page.

One user's locations

GET /v2/users/:user_id/locations

Returns location history for one user.

Use it for:

- Individual activity details.
- Returning-student calculations.
- Debugging incorrect session totals.
- Student-specific history.

Suggested refresh: On demand

Global locations

GET /v2/locations

Returns location records globally, subject to access and filters. The campus-specific location endpoint is preferred for the Warsaw dashboard to reduce unnecessary data.

One location record

GET /v2/locations/:id

Returns one location-session record. Used mainly for debugging or inspecting a specific session.

3.6 Projects Users

All project-user records

GET /v2/projects_users

Returns project-user records for students who are working on or have completed projects. Important fields may include the user, project, status, final mark, validation result, and creation and update times.

Use it for:

- Recently passed projects.
- Project completion rankings.
- Failures and retries.
- Project popularity.
- Students currently working on projects.

Suggested refresh: Every 5-15 minutes for recent activity

One user's projects

```
GET /v2/users/:user_id/projects_users
```

Returns project records for one student.

Use it for:

- Student progress pages.
- Recently completed projects.
- Project history.
- Debugging duplicate or versioned

Suggested refresh: On demand or every few hour Users on one project

Users on one project

```
GET /v2/projects/:project_id/projects_users
```

Returns users associated with one project.

Use it for:

- Number of students working on a project.
- Project-specific completion statistics.
- Pass and failure analysis.
- One project-user relationship

One project-user relationship

GET /v2/projects_users/:id

Returns one project-user relationship. Used for detailed debugging and record inspection.

3.7 Projects

All projects

GET /v2/projects

Returns project definitions.

Use it for:

- Project ID.
- Project name.
- Parent project.
- Project metadata.
- Mapping project-user records to readable project names.

Suggested refresh: Daily

Projects in one cursus

GET /v2/cursus/:cursus_id/projects

Returns projects associated with one cursus. Used to restrict project analytics to the selected curriculum.

Suggested refresh: Daily

One project

GET /v2/projects/:id

Returns detailed information about one project. Used when project-version or project-session information requires closer inspection.

3.8 Project Sessions

Sessions for one project

GET /v2/projects/:project_id/project_sessions

Returns the sessions or configurations associated with a project.

Use it for:

- Distinguishing different project versions.
- Campus-specific project configurations.
- Cursus-specific project configurations.

Suggested refresh: Daily

One project session

GET /v2/project_sessions/:id

Returns one project-session definition. Used when normalizing project versions.

3.9 Scale Teams and Evaluations

All evaluation records

GET /v2/scale_teams

Returns evaluation or defence records. Possible information includes the evaluated team, corrector, evaluated users, project, evaluation status, dates, and final result where available.

Use it for:

- Top evaluators.
- Most evaluated students.
- Evaluation count.
- Evaluation heatmap.
- Most evaluated Projects.
- Recent completed evaluations.

Suggested refresh: Every 10-30 minutes

Evaluations given by a user

GET /v2/users/:user_id/scale_teams/as_corrector

Returns scale teams where the selected user acted as the corrector.

Use it for:

- Number of evaluations conducted.
- Evaluator-specific history.
- Verifying the top-evaluator leaderboard.
- Evaluations received by a user

Evaluations received by a user

GET /v2/users/:user_id/scale_teams/as_corrected

Returns scale teams where the selected user was evaluated.

Use it for:

- Number of evaluations received.
- Most-evaluated-student leaderboard.
- Student evaluation history.

Evaluations for one project

GET /v2/projects/:project_id/scale_teams

Returns evaluation records for one project.

Use it for:

- Most evaluated projects.
- Evaluation activity by project.
- Project-specific evaluation patterns.

One evaluation record

GET /v2/scale_teams/:id

Returns one evaluation record. Used for detailed inspection and debugging.

3.10 Coalitions

All coalitions

GET /v2/coalitions

Returns coalition definitions and current coalition information.

Use it for:

- Coalition names.
- Coalition IDs.
- Coalition logos or images.
- Current standings where available.

Suggested refresh: Hourly

Coalitions in one bloc

GET /v2/blocs/:bloc_id/coalitions

Returns coalitions belonging to one bloc. Used when the Warsaw coalition set is organized under a specific bloc.

A user's coalition

GET /v2/users/:user_id/coalitions

Returns the coalition membership of a user.

Use it for:

- Displaying the student's coalition.
- Joining students with coalition logos and colors.

Suggested refresh: Daily or cached

3.11 Coalition Users

All coalition-user relationships

GET /v2/coalitions_users

Returns relationships between users and coalitions.

Use it for:

- Coalition membership.

- Associating users with a coalition.
- Contribution fields only when the response, exposes a reliable value.

Suggested refresh: Hourly or daily

Members of one coalition

GET /v2/coalitions/:coalition_id/coalitions_users

Returns members of one coalition. Used for coalition-specific member lists.

One user's coalition record

GET /v2/users/:user_id/coalitions_users

Returns coalition-user records for one student. Used to verify membership and available contribution fields.

4. Endpoint-to-Feature Mapping

This section maps every dashboard feature to the endpoints that supply its data, the processing logic applied by the backend, and whether historical storage is required.

Feature	Main endpoints	Historical storage?
Current online students	/v2/campus/:campus_id/locations	No
Cluster occupancy	/v2/campus/:campus_id/locations	Required for past data
Weekly campus-time leaderboard	/v2/campus/:campus_id/locations	Recommended
Returning students	/v2/users/:user_id/locations	Required (partial)
Night Owls	/v2/campus/:campus_id/locations	Recommended
Early Birds	/v2/campus/:campus_id/locations	Recommended
Recently passed projects	/v2/projects_users, /v2/projects	Recommended
Top evaluators	/v2/scale_teams, .../as_corrector	Recommended
Most evaluated students	/v2/scale_teams, .../as_corrected	Recommended
Evaluation activity heatmap	/v2/scale_teams	Required
Coalition leaderboard	/v2/coalitions	Required for growth
Weekly coalition contributors	/v2/coalitions_users	Required
Black Hole status	/v2/cursus/:cursus_id/cursus_users	Not required

Current Online Students

GET/v2/campus/:campus_id/locations

Returns all current campus location sessions.

Use it for:

- Current online students.
- A session is active when end_at is null.
- Historical storage: No, for the current view.

Cluster Occupancy

GET /v2/campus/:campus_id/locations

Returns all current campus location sessions.

Use it for:

- Reading the workstation hostname.
- Grouping hosts by cluster.
- Treating active sessions as occupied.
- Returning occupied and free seat counts.
- Historical storage: Required for past occupancy and seat heatmaps.

Weekly Campus-Time Leaderboard

GET /v2/campus/:campus_id/locations

Returns campus location sessions.

Use it for:

- Selecting sessions overlapping the week.
- Calculating session duration.
- Trimming sessions at the reporting boundaries.
- Grouping duration by user.
- Sorting users in descending order.

Returning Students

GET /v2/users/:user_id/locations

Returns a user's campus location history.

Use it for:

- Finding the latest session.
- Finding the previous session.
- Calculating the inactivity gap.
- Applying a configurable threshold such as 14 days.

Historical storage: Required when the API does not return enough old sessions.

Night Owls

GET /v2/campus/:campus_id/location

Returns campus location sessions.

Use it for:

- Calculating session overlap with the night window.
- Grouping night duration by user.
- Time window: 22:00–06:00 Europe/Warsaw.
- Historical storage: Recommended.

Early Birds

GET /v2/campus/:campus_id/locations

Returns campus location sessions.

Use it for:

- Calculating session overlap with the morning window.
- Grouping morning duration by user.
- Time window: 06:00–10:00 Europe/Warsaw.

- Historical storage: Recommended.

Recently Passed Projects

GET /v2/projects_users
GET /v2/projects

Returns project submissions and project information.

Use it for:

- Confirming validated? = true.
- Joining project and user information.
- Sorting by the best reliable completion timestamp.

Historical storage: Recommended for long-term trends.

Top Evaluators

GET /v2/scale_teams
GET /v2/users/:user_id/scale_teams/as_corrector

Returns completed evaluation records.

Use it for:

- Including completed evaluations only.
- Identifying the corrector.
- Grouping evaluations by corrector.
- Counting evaluations in the selected period.

Historical storage: Recommended.

Most Evaluated Students

GET /v2/scale_teams
GET /v2/users/:user_id/scale_teams/as_corrected

Returns completed evaluation records.

Use it for:

- Including completed evaluations only.
- Identifying evaluated users.
- Grouping evaluations by student.
- Counting evaluations in the selected period.
- Historical storage: Recommended.

Evaluation Activity Heatmap

GET /v2/scale_teams

Returns completed evaluation records.

Use it for:

- Converting completion times to Europe/Warsaw.
- Grouping evaluations by weekday and hour.
- Counting completed evaluations

Historical storage: Required for reliable weekly and monthly analysis.

Coalition Leaderboard

GET /v2/coalitions

Returns coalition information and current standings.

Use it for:

- Displaying current coalition scores.

- Sorting coalitions by current score.
- Historical storage: Required for weekly score growth.

Weekly Coalition Contributors

GET /v2/coalitions_users
GET /v2/users/:user_id/coalitions_users

Returns coalition membership information.

Use it for:

- Using timestamped individual contribution data when available.
- Comparing stored per-user snapshots when contribution data is unavailable.
- Historical storage: Required unless the API provides contribution events.

5. Endpoint-to-Feature Mapping

The backend must avoid calling the 42 API every time the TV frontend refreshes. The frontend requests processed data from the backend, and the backend uses caching and scheduled jobs so that the 42 API is only contacted as often as necessary.

Refresh Strategy Table

Data	Refresh method	Suggested interval	Reason
OAuth token	Automatic	Before expiry	Avoid unnecessary token requests
Live locations	Polling job	1-5 minutes	Occupancy changes frequently
Dashboard overview	Cache refresh	1-5 minutes	Depends on live locations
Recent project passes	Scheduled polling	5-15 minutes	Near-real-time but not instantaneous
Evaluation records	Scheduled polling	10-30 minutes	Moderate update frequency
Coalition scores	Hourly cron	1 hour	Scores do not require minute-level polling
Cursus-user data	Scheduled polling	6-12 hours	Levels and Black Hole dates change less frequently
User profiles	Cache	12-24 hours	Images and basic metadata are mostly static
Project definitions	Daily cron	24 hours	Reference data changes infrequently
Campus definitions	Daily or weekly	24 hours or longer	Mostly static
Historical snapshots	Daily cron	Once per day	Needed for trends and comparisons

Protections

- Cache responses.
- Fetch all paginated results through controlled sequential or limited-concurrency requests.
- Avoid requesting every user's detailed data on every refresh.
- Use batch or collection endpoints whenever possible.
- Respect rate-limit response headers when available.
- Pause requests when receiving HTTP 429.
- Use exponential backoff.
- Add random delay or jitter to retries.
- Keep the latest successful dataset available.

Example Schedule

Live location job: Every 3 minutes Coalition snapshot: Every hour Daily analytics snapshot: Every day at 00:10 Europe/Warsaw

6. Endpoint-to-Feature Mapping

Historical Data Storage

Some dashboard metrics cannot be calculated accurately from a single current API response. To support these features, the backend stores timestamped records for:

- Location sessions
- Seat occupancy
- Student campus-time totals
- Coalition scores
- Individual coalition contributions when available
- Project completions
- Evaluation records
- Daily campus summaries
- Weekly leaderboard positions

Calculations

Weekly coalition gain:

Weekly coalition gain = current coalition score - score stored seven days earlier

Weekly student contribution:

Weekly student contribution = current contribution value - stored contribution value

12 Seat usage last week:

Seat usage last week = sum of occupied session duration for that workstation

Weekly campus time:

Weekly campus time = sum of each student's session overlap with the selected week

7. Error Handling

This section describes what happens when the 42 API is unavailable or returns an error.

7.1 API Unavailable

The backend should:

- Stop the failed data-processing operation safely.
- Keep the last successful cached response.
- Return cached data to the frontend.
- Include the last successful update time.
- Retry later using exponential backoff.
- Log the technical error without exposing secrets.

Frontend message:

- Live data is temporarily unavailable.
- Showing cached data from 14:20.

7.2 Expired Access Token

- Detect HTTP 401.
- Request a new token.
- Retry the original request once.
- If it fails again, return an error.
- Never retry continuously.

7.3 Rate Limit Exceeded

- Detect HTTP 429.
- Read retry information when provided.
- Pause outgoing requests.
- Retry after a delay.
- Serve cached data during the delay.

7.4 Invalid Filter

The API may reject unsupported filters.

Handling:

- Validate filters before sending the request.
- Do not use undocumented filters such as filter[active] when unsupported.
- Log the endpoint and rejected parameters.
- Fall back to retrieving data and filtering safely in the backend when reasonable.

Example:

Do not send: filter[active]=true unless the endpoint explicitly supports the active field.

7.5 Pagination Failure

- Keep already retrieved valid pages only for debugging.
- Do not publish incomplete analytics as complete results.
- Mark the collection job as incomplete.
- Continue serving the previous complete cached dataset.

7.6 Partial Service Failure

- If locations fail but projects succeed:
- Keep the project section available.
- Show cached cluster data.
- Do not make the entire dashboard unusable.

Example:

```
{
  "status": "partial",
  "sections": {
    "projects": "fresh",
    "clusters": "cached",
    "coalitions": "fresh"
  }
```

7.7 Invalid or Missing Fields

The application must never display:

- undefined
- NaN
- Invalid Date
- null hours

Safe fallbacks should be used instead:

- Not available
- Unknown student
- No image
- Active session

8. References

- 42 API Documentation — https://api.intra.42.fr/apidoc
- 42 API Guides — https://api.intra.42.fr/apidoc/guides
- OAuth 2.0 Specification — https://oauth.net/2/
- 42 Warsaw Campus — https://42warsaw.pl